



SKU LIKELION
UX/UI TRACK

UX/UI Team
Official LIKELION at SKU

WEEK 4.

기획 & 디자인



노태경 김지유



4주차 기획시간!

길을 잃지 않게 하는 설계

User Flow / IA / Happy & Unhappy Path

화면을 만들기 전에
사용자가 **목표에 도달하는 길**부터 설계해요.



지난주 과제 확인!

내 주제 **MVP** 최종본 제작!

핵심 문제 1문장, 타겟 1문장, What 1문장, MVP 1문장, 이번에 만들 것 3개 이하,
이번에는 만들지 않을 것 3개 이하, 성공 기준 1줄,
실습에서 받은 피드백과 반영 내용 2줄

세션 시작 전 ! 간단한 이야기 나누기~~

지금까지 주제를 정하고 실습을 하며 어떻게
가장 힘들고 고민됐는지 서로 얘기 나눠봐요



WHY

문제정의
PainPoint



WHO

누가 쓰는가?
Target



WHAT

무엇을 만들 것인가?
MVP





오늘 배우게 될 것

구조 설계의 핵심을 배우고
직접 서비스의 지도 그려보기

1

User Flow & IA

구조 설계와 사용자 이동 흐름 이해

2

핵심 흐름 & 예외 흐름

Happy Path & Unhappy Path

3

FigJam 사용하기

IA & User Flow 제작

흐름이 설계다

세련된 UI는 그 다음입니다.

사용자가 막힘없이 목적지에 도달하는
논리적인 지도를 먼저 만들어야 합니다.



서비스 기획의 순서

오늘 배울 것!



WHY

문제정의
PainPoint



WHO

누가 쓰는가?
Target



WHAT

무엇을 만들 것인가?
MVP



HOW

어떻게 구현할 것인가?
IA & UserFlow



HOW "어떤 방식으로 구체화하나요?" (구조와 흐름 설계)

사용자가 목표를 이루기까지의 **핵심 흐름** 정리하기

필요한 화면과 메뉴를 **구조로 정리하기**

핵심 성공 경로를 **IA와 User Flow**로 구체화하기

팀이 같은 그림을 보도록 **산출물로 정리하기**

같은 화장실, 다른 질문. 어떤 차이점이 있을까요?



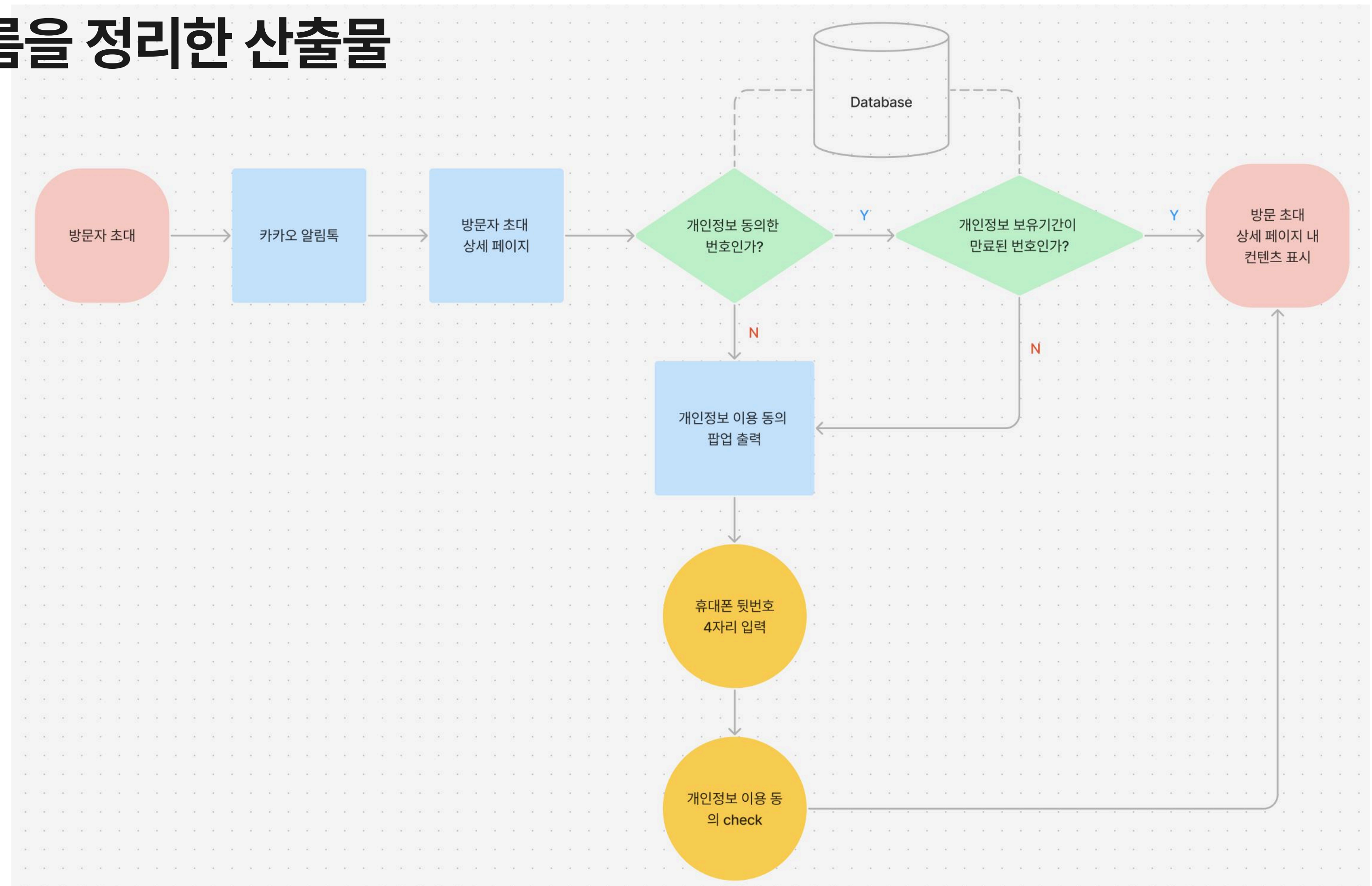


User Flow

제품 안에서 사용자가 특정 목표를 이루기 위해 거치는
전형적이거나 이상적인 단계의 흐름을 정리한 산출물

특징

- 과업 중심: 하나의 목표에 집중
- 제품 내부 흐름: 짧고 명확한 경로





User Flow는 단순 문서 작업이 아니라 협업 비용을 줄이는 도구입니다

팀이 **같은 그림**을 보게 해요

- 기획, 디자인, 개발의 기준이 하나로 정리됩니다

빠진 분기를 **먼저 발견**해요

- 실패 / 권한 / 없음 케이스를 초기에 점검합니다

질문이 줄어들어요

- 개발 중 추가 확인이 필요한 구간을 줄이는 데 도움이 됩니다

분기 : 흐름이 한 방향으로만 가지 않고 조건에 따라 길이가 나뉘는 지점



송금 User Flow 예시





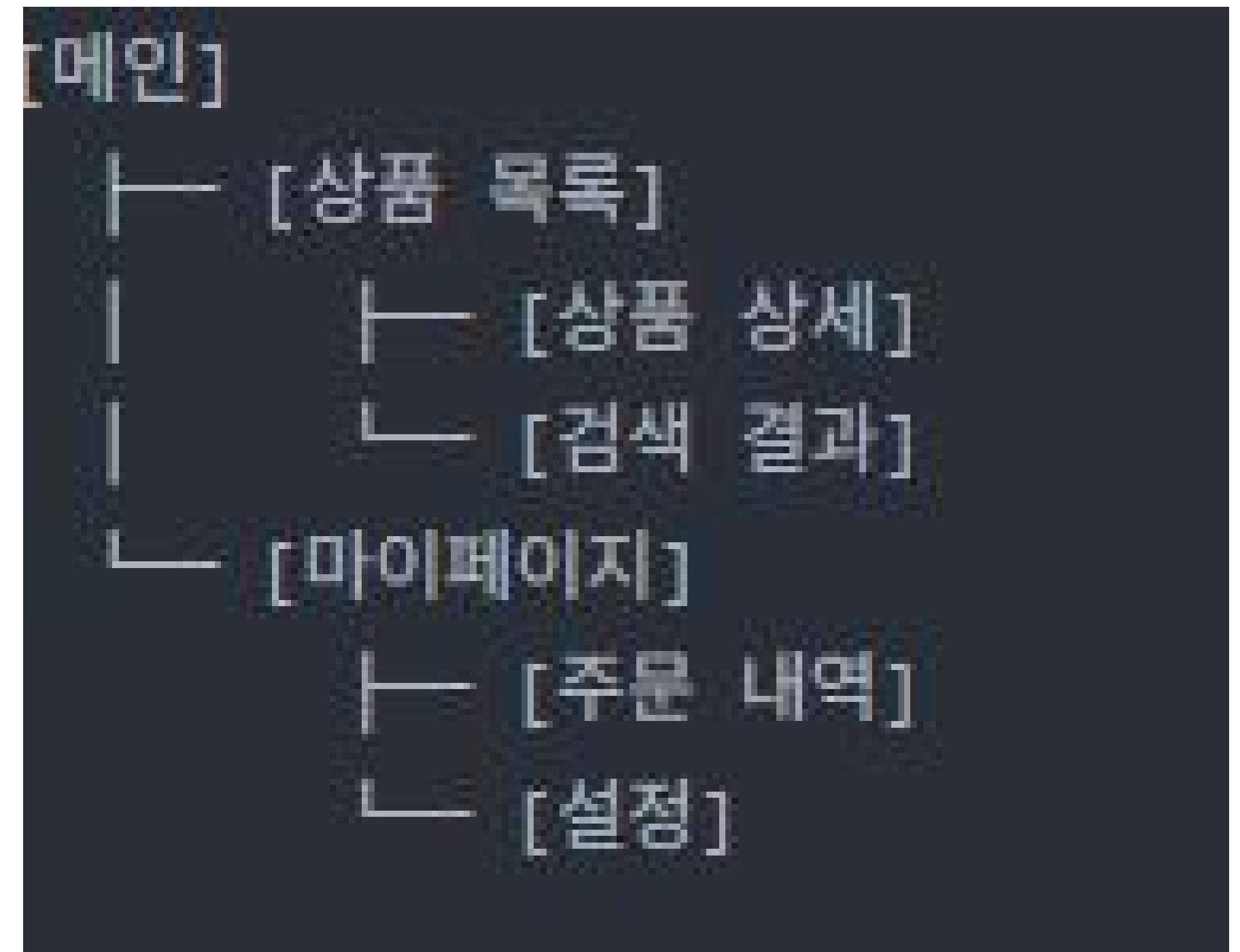
IA (Information Architecture, 정보 구조도)

서비스의 뼈대와 정보 구조를 정리하는 산출물

메뉴의 위계(Hierarchy), 화면의 소속 관계, 정보의 깊이를 정리해 사용자가 길을 잃지 않도록 돕습니다.

특징

- 전체 구조를 한눈에 볼 수 있습니다.
- 화면과 메뉴의 소속 관계를 정의합니다.
- UI가 아니라 UI를 떠받치는 구조입니다.



정보 구조 (IA) 예시: 사이트맵



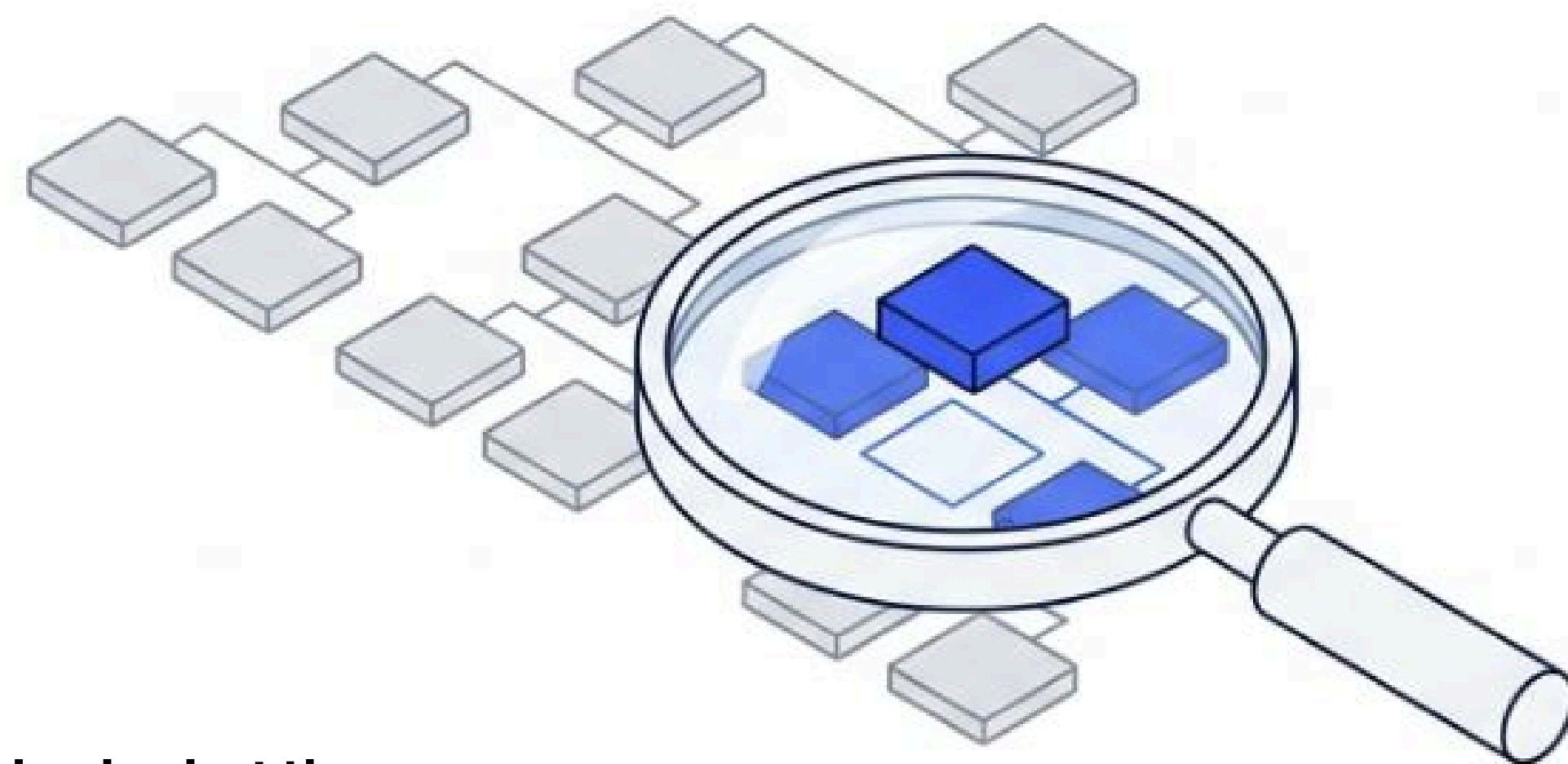
IA를 보는 관점

빠짐 없이 채웠는가?

모든 페이지가 IA 어딘가에 소속되어 있어야 해요

찾기 쉬운가?

사용자가 원하는 메뉴를 직관적인 위치에서 찾을 수 있어야 해요



건물 안내도 VS 내비게이션



공간이 어떻게 구성되어 있는지

IA (Information Architecture)

- 공간(Space) 중심
- 정적(Static)
- 구조적(Structural)



User Flow

- 시간(Time) 중심
- 동적(Dynamic)
- 절차적(Procedural)

시작점에서 끝점까지
어떻게 이동하는지

같은 화장실, 다른 질문. 어떤 차이점이 있을까요?



순서보다 **역할의 차이**가 중요해요!!

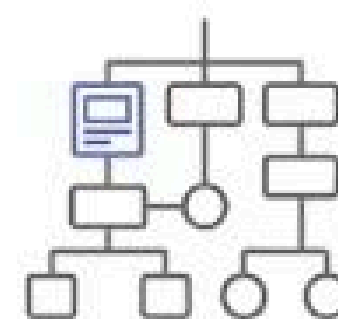
그래서... 뭘부터 해야 할까요?

과업 중심



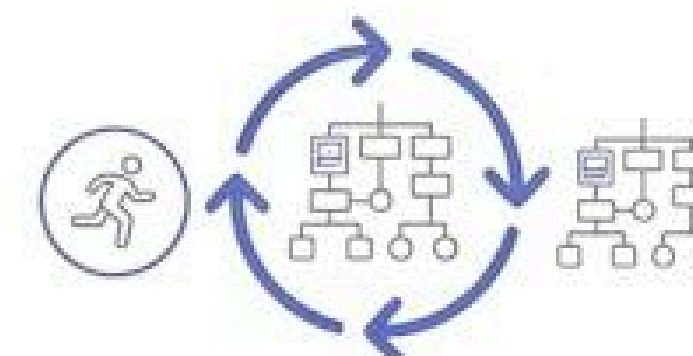
👉 User Flow 먼저

구조 중심



👉 IA 먼저

실제 작업



👉 실제 작업에서는 둘 다 반복

정해진 순서는 없고,
상황에 따라 선택하거나 반복합니다.



Happy Path vs. Unhappy Path

완벽한 설계의 조건



우리가 바라는 **이상적인 경로**



우리가 마주할 **현실적인 경로**

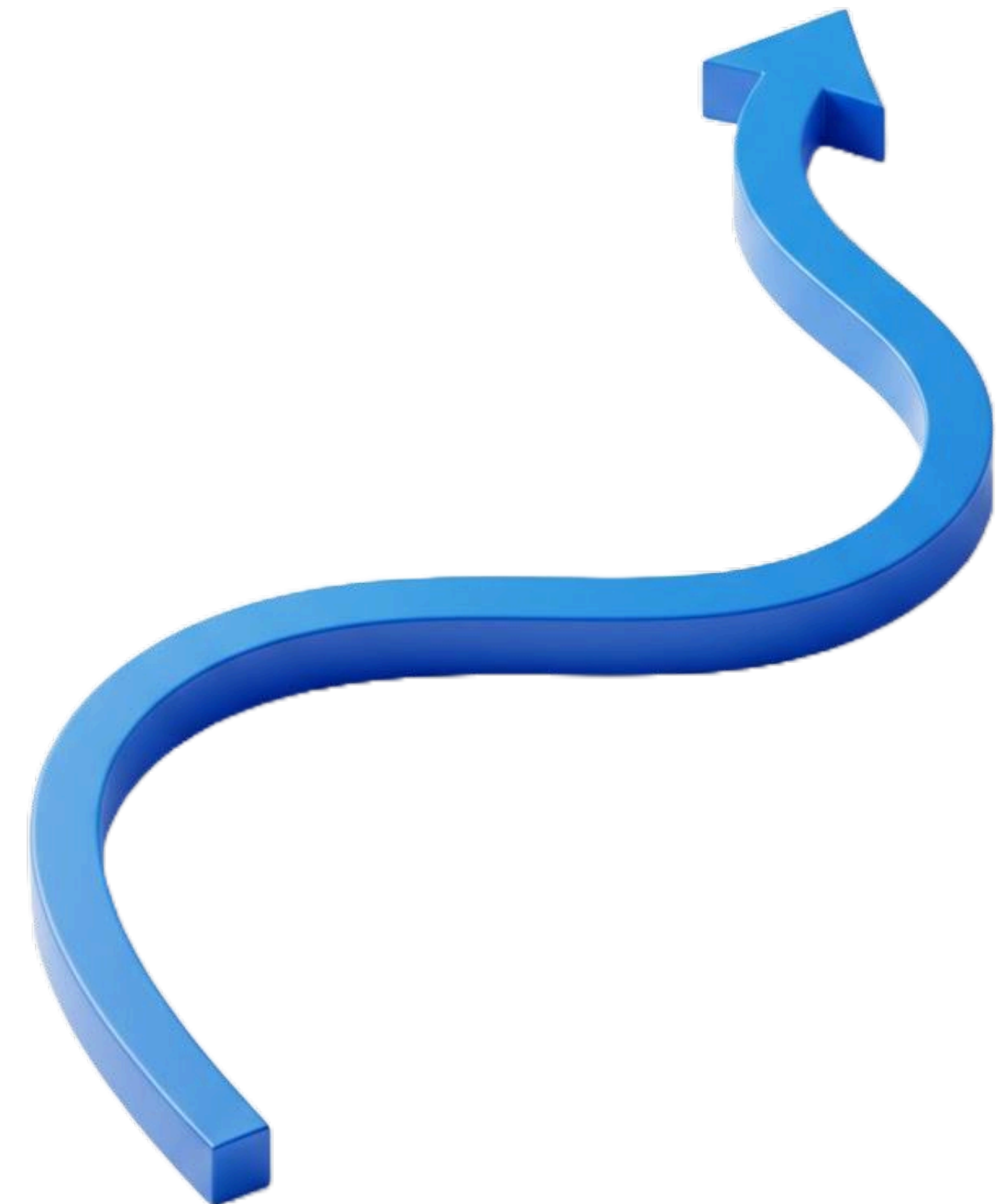
서비스에는 언제나 두가지 길이 존재해요



Happy Path

가장 전형적이거나 이상적인 성공 경로

사용자가 특정 목표를 달성할 때
막힘 없이 거치게 되는 대표적인 성공 흐름



Happy Path: 실수 없는 성공 경험

가장 이상적인 시나리오 (로그인 성공 예시)



하지만... 현실은...다릅니다...

서비스는 우리의 예상대로 움직이지 않아요.



#시스템 오류

#데이터의 부재

#사용자의 실수





Unhappy Path

오류·예외·대안 경로 점검하기

목표 달성을 방해하거나 흐름을 바꾸는
오류, 예외, 대안 상황을 함께 점검하는 관점입니다



Unhappy Path를 함께 보는 이유

구현의 중단 방지

실패 시 화면이나 재시도 규칙이 비어 있으면 개발이 멈춰요

비용 절감

테스트 단계에서 문제를 발견하면
처음 설계할 때 고치는 것보다 훨씬 돈과 시간이 많이 들어요

예외 설계가 안되면.. 개발이 계속 미뤄지고..
PM, 디자이너에게 개발자는 계속 물어보고..
이런 과정을 거치면서 다툼이 생깁니다 😓

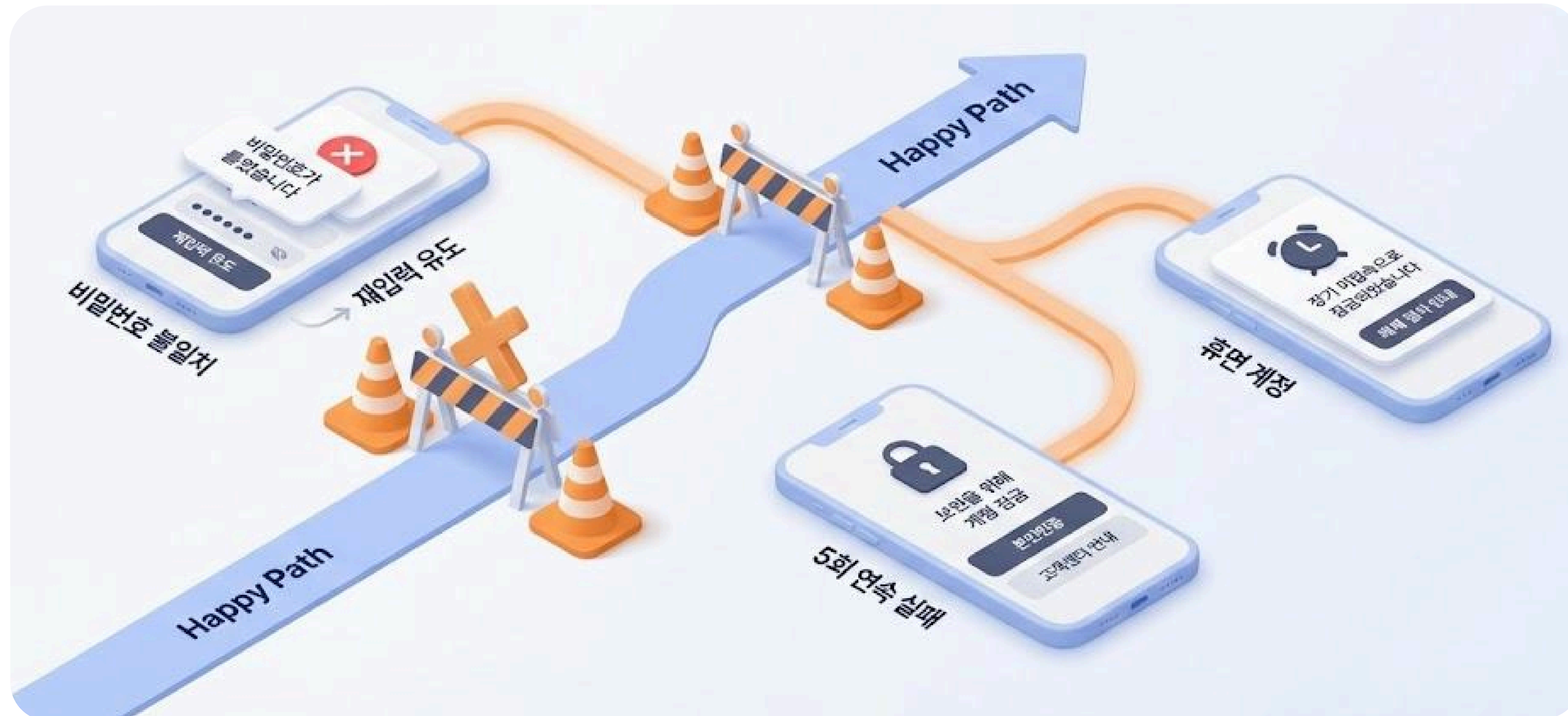


예외를 늦게 발견할수록 비용이 커져요



정확한 숫자보다 중요한 것은
 늦게 고칠수록 수정 범위와 커뮤니케이션 비용이 커진다는 점입니다.

Unhappy Path 예시: 로그인 실패



오류를 알려주고
다시 시도하거나 복구할 수 있게 설계해야 해요!

Unhappy Path를 볼 때 점검할 6가지 질문



실패 시

어디로 돌아가는가



입력값 공백

무엇을 막아야 하는가



결과 0건

무엇을 보여주는가



네트워크 끊김

상태를 어떻게 알리나



비회원/로그아웃

어디까지 허용하나



중복/반복 행동

어떻게 처리하나